

React.js



PRISMA

find()

`prisma.user.findUnique()`

Get record by ID or unique identifier

```
// By unique identifier
const user = await prisma.user.findUnique({
    where: { email: 'elsa@prisma.io', },
})

// By ID
const user = await prisma.user.findUnique({
    where: { id: 99, },
})
```

`prisma.user.findMany()`

Get all records

```
const users = await prisma.user.findMany()
```

`prisma.user.findFirst()`

Get the first record that matches a specific criteria

```
const users = await prisma.user.findFirst()
```

https://www.prisma.io/docs/orm/prisma-client/queries

filtered list of records

```
prisma.user.findMany({
   where:{  conditions   }
})
```

Filter by a single field value

```
const result = await prisma.customers.findMany({
    where: { country:"France", }
});
```

Filter by multiple fields value

```
const result = await prisma.customers.findMany({
    where: {
        country: "France",
        contactFirstName: { startsWith: 'E', }
    }
});
```

Implicit AND

Select subset fields

```
const result = await prisma.customers.findMany({
    where: {
        country: "France",
        contactFirstName: { startsWith: 'M', }
    },
    select: {
        contactFirstName: true,
        city: true,
    },
});
```


```
[
  {
    "contactFirstName": "Mary ",
    "city": "Lyon"
  },
  {
    "contactFirstName": "Martine ",
    "city": "Lille"
  },
  {
    "contactFirstName": "Marie",
    "city": "Paris"
  }
]
```

Comparison Operators

gt, gte, lt, lte, not

```
const result = await prisma.products.findMany({
    where: { buyPrice: { gt: 100 } }
})
```

```
const result = await prisma.products.findMany({
    where: {
        buyPrice: {
            gt: 50,
            lt: 60
        }
    },
})
```

In / not in

in notIn

```
const result = await prisma.customers.findMany({
    where: {
        country: {
            in: ['France', 'Germany']
        },
    },
})
```

And OR

```
prisma.user.findMany({
   where:{
      AND:[ {cond1}, [cond2], ... ]
   }
})
```

Explicit AND

```
await prisma.user.findMany({
    where: {
        AND: [
            { name: 'John Doe' },
            { age: { gt: 25 } },
        ],
    },
```

OR

```
await prisma.user.findMany({
    where: {
        OR: [
            { name: 'John Doe' },
            { email: { endsWith: 'company.com' } },
        ],
    },
```

```
const users = await prisma.user.findMany({
  where: {
    OR: [
      {
        name: { startsWith: 'E’, },
      },
      {
        AND: {
          profileViews: { gt: 0, },
          role: { equals: 'ADMIN’, },
        },
      },
    ],
  },
})
```

contains, startsWith, endsWith

```
const result = await prisma.customers.findMany({
    where: {
        country: "France",
        contactFirstName: { startsWith: 'M', }
    },
});
```

```
const result = await prisma.customers.findMany({
    where: {
        country: "France",
        contactFirstName: {
            startsWith: 'M',
            contains: 'ry'
        }
    },
});
```

Include related records

customers + orders

```
const result = await prisma.customers.findMany({
    where: {
        country: "France",
        contactFirstName: { startsWith: 'M', }
    },
    include: {
        orders: true,
    },
});
```

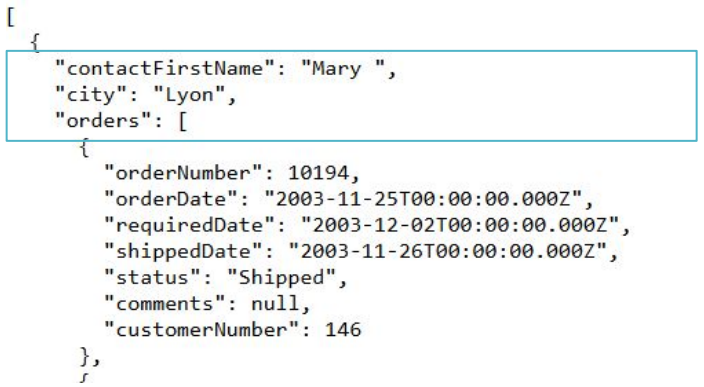

```
[
  {
    "customerNumber": 146,
    "customerName": "Saveley & Henriot, Co.",
    "contactLastName": "Saveley",
    "contactFirstName": "Mary ",
    "phone": "78.32.5555",
    "addressLine1": "2, rue du Commerce",
    "addressLine2": null,
    "city": "Lyon",
    "state": null,
    "postalCode": "69004",
    "country": "France",
    "salesRepEmployeeNumber": 1337,
    "creditLimit": 123900,
    "orders": [
      {
        "orderNumber": 10194,
        "orderDate": "2003-11-25T00:00:00.000Z",
        "requiredDate": "2003-12-02T00:00:00.000Z",
        "shippedDate": "2003-11-26T00:00:00.000Z",
        "status": "Shipped",
        "comments": null,
        "customerNumber": 146
      },
      {
        "orderNumber": 10208,
        "orderDate": "2004-01-02T00:00:00.000Z",
        "requiredDate": "2004-01-11T00:00:00.000Z",
        "shippedDate": "2004-01-04T00:00:00.000Z",
        "status": "Shipped",
        "comments": null,
        "customerNumber": 146
      },
      {
        "orderNumber": 10227,
        "orderDate": "2004-03-02T00:00:00.000Z",
        "requiredDate": "2004-03-12T00:00:00.000Z",
        "shippedDate": "2004-03-08T00:00:00.000Z",
        "status": "Shipped",
        "comments": null,
        "customerNumber": 146
      }
    ]
  },
  {
    "customerNumber": 171,
```

customers + orders + orderdetails + products

```
const result = await prisma.customers.findMany({
    where: {
        country: "France",
        contactFirstName: { startsWith: 'M', }
    },
    include: {
        orders: {
            include: {
                orderdetails: {
                    include: {
                        products: true,
                    },
                },
            },
        },
    },
});
```

Include with select

```
const result = await prisma.customers.findMany({
    where: {
        country: "France",
        contactFirstName: { startsWith: 'M', }
    },
    select: {
        contactFirstName: true,
        city: true,
        orders:true,
    },
});
```



```
const result = await prisma.customers.findMany({
    where: {
        country: "France",
        contactFirstName: { startsWith: 'M', }
    },
    select: {
        contactFirstName: true,
        city: true,
        orders: {
            select: {orderDate: true,},
        },
    },
});
```


```
[
  {
    "contactFirstName": "Mary ",
    "city": "Lyon",
    "orders": [
      {
        "orderDate": "2003-11-25T00:00:00.000Z"
      },
      {
        "orderDate": "2004-01-02T00:00:00.000Z"
      },
      {
        "orderDate": "2004-03-02T00:00:00.000Z"
      }
    ]
  },
  {
    "contactFirstName": "Martine ",
```

Relation Filters

```
const result = await prisma.orders.findMany({
    where: {
        status: "Shipped",
        customers:{
            is:{
                customerName :{startsWith:'M'}
            }
        }
    },
});
```

orders



orders




```
[
  {
    "orderNumber": 10113,
    "orderDate": "2003-03-26T00:00:00.000Z",
    "requiredDate": "2003-04-02T00:00:00.000Z",
    "shippedDate": "2003-03-27T00:00:00.000Z",
    "status": "Shipped",
    "comments": null,
    "customerNumber": 124
  },
  {
    "orderNumber": 10135,
    "orderDate": "2003-07-02T00:00:00.000Z",
    "requiredDate": "2003-07-12T00:00:00.000Z",
    "shippedDate": "2003-07-03T00:00:00.000Z",
    "status": "Shipped",
    "comments": null,
    "customerNumber": 124
  },
  {
    "orderNumber": 10142,
```




```
    include : {
        customers:true,
    }
```

orders + customers

Sorting

```
const result = await prisma.orders.findMany({
    orderBy: { orderDate:'asc' }
});
```

Sort by relation

```
const result = await prisma.orders.findMany({
    orderBy: {
        customers: { city: 'desc' }
    }
});
```

```
const result = await prisma.customers.findMany({
    where: {
        country: "France",
        contactFirstName: { startsWith: 'M', }
    },
    select: {
        contactFirstName: true,
        city: true,
        orders: {
            select: { orderDate: true, },
            orderBy: { orderDate: 'desc' }
        },
    },
    orderBy: {
        city: 'asc'
    }
});
```



Aggregate

```
const aggregations = await prisma.products.aggregate({

    _avg: {
        buyPrice: true,
        quantityInStock: true,
    },

    _count: {
        buyPrice: true,
        quantityInStock: true,
    },

    _sum: {
        quantityInStock: true,
    }
})
```

```
{
    _avg: {
        buyPrice: 54.39518181818183,
        quantityInStock: 5046.6455
    },
    _count: {
        buyPrice: 110,
        quantityInStock: 110
    },
    _sum: {
        quantityInStock: 555131.005
    }
```

```
const customersWithOrdersCount = await prisma.customers.findMany({
    select: {
        customerName: true,
        city: true,
        _count: {
            select: { orders: true },
        },
    },
})

res.send(customersWithOrdersCount)
```

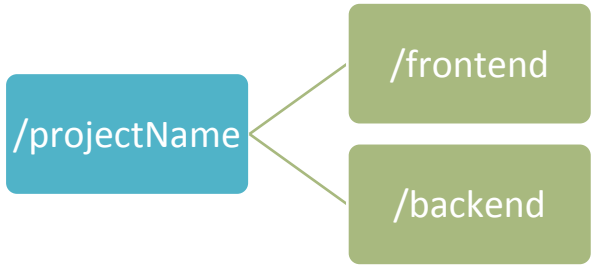


```
[
  {
    "customerName": "Atelier graphique",
    "city": "Nantes",
    "_count": {
      "orders": 3
    }
  },
  {
    "customerName": "Signal Gift Stores",
    "city": "Las Vegas",
    "_count": {
      "orders": 3
    }
  },
  {
```



Authentication

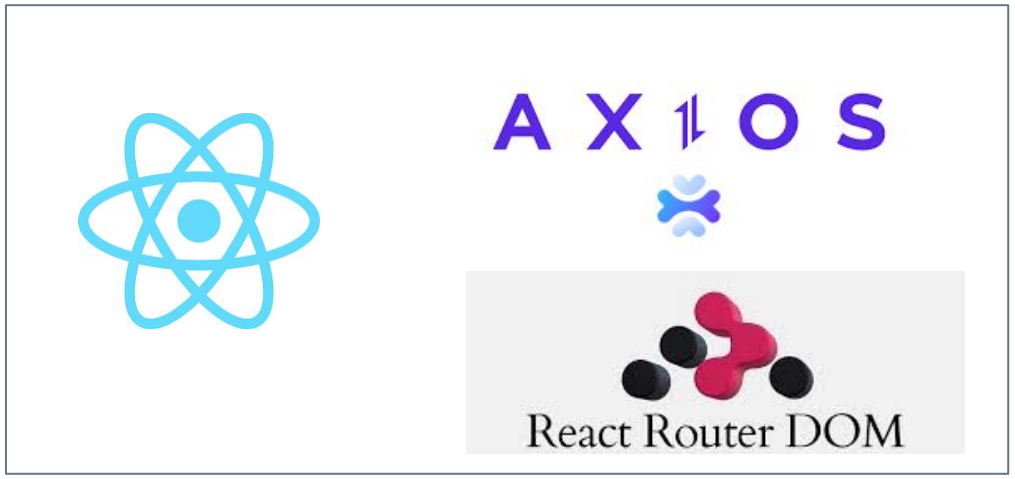
Create project



FRONT-END

/frontend

Install ReactJs inside client



npm install [dependency]

BACK-END

/backend

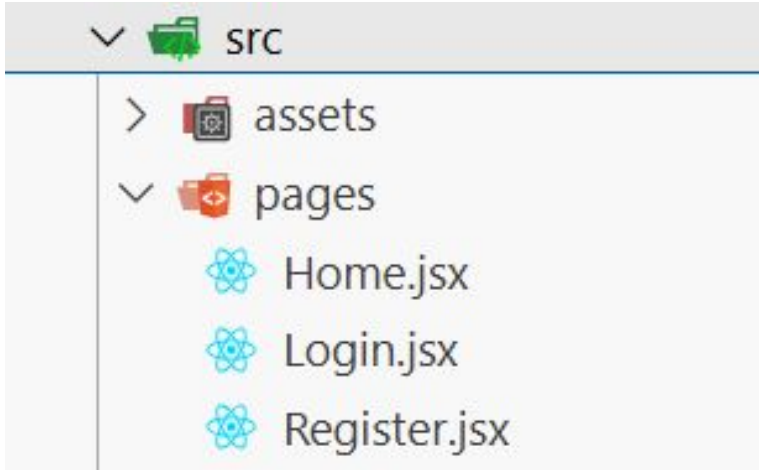
>npm init –y

Install dependencies



Cores - Cross-Origin Resource Sharing

Register and Login




```
function Register() {
  return (
    <>
      <h1>Register Page</h1>
    </>
  )
}
```

```
export default Register
```

```
function Login() {
  return (
    <>
      <h1>Login Page</h1>
    </>
  )
}
```

```
export default Login
```

```
function Home() {
  return (
    <>
      <h1>Home page</h1>
    </>
  )
}
```

```
export default Home
```

App.jsx

```jsx
import { BrowserRouter, Routes, Route } from "react-router-dom"
import Register from './pages/Register'
import Login from './pages/Login'
import Home from './pages/Home'

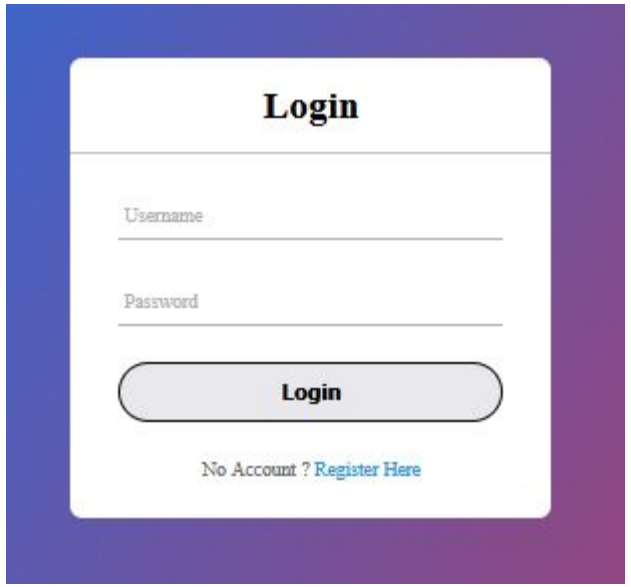
function App(){
  return(
    <BrowserRouter>
        <Routes>
            <Route path="/" element={<Home/>}></Route>
            <Route path="/login" element={<Login/>}></Route>
            <Route path="/register" element={<Register/>}></Route>
        </Routes>
    </BrowserRouter>
  )
}

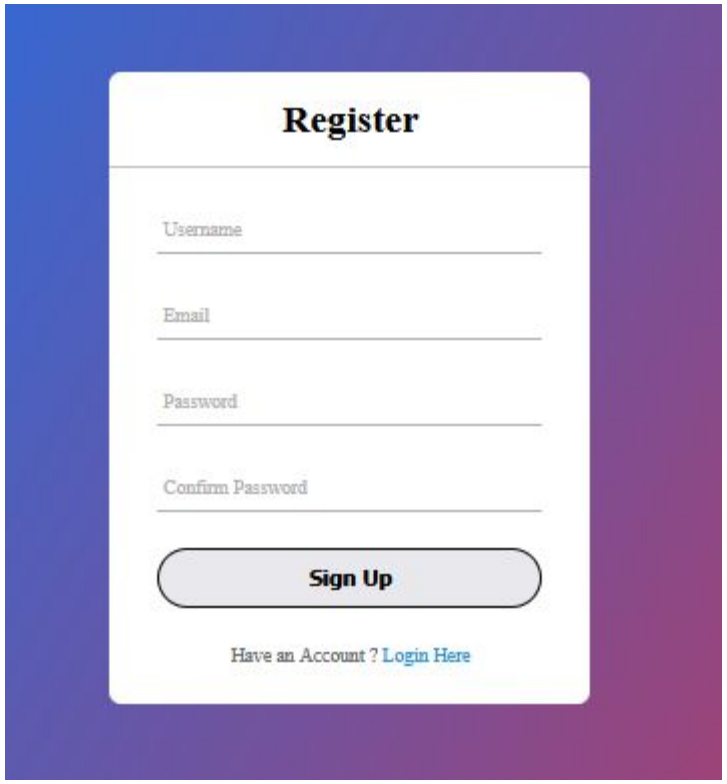
export default App
```

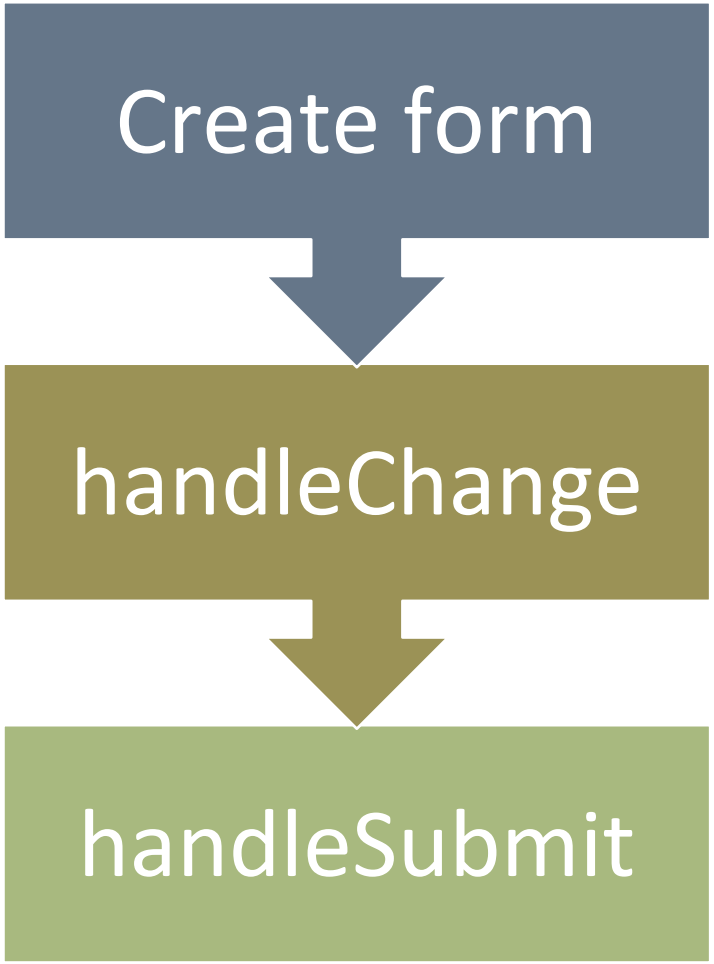


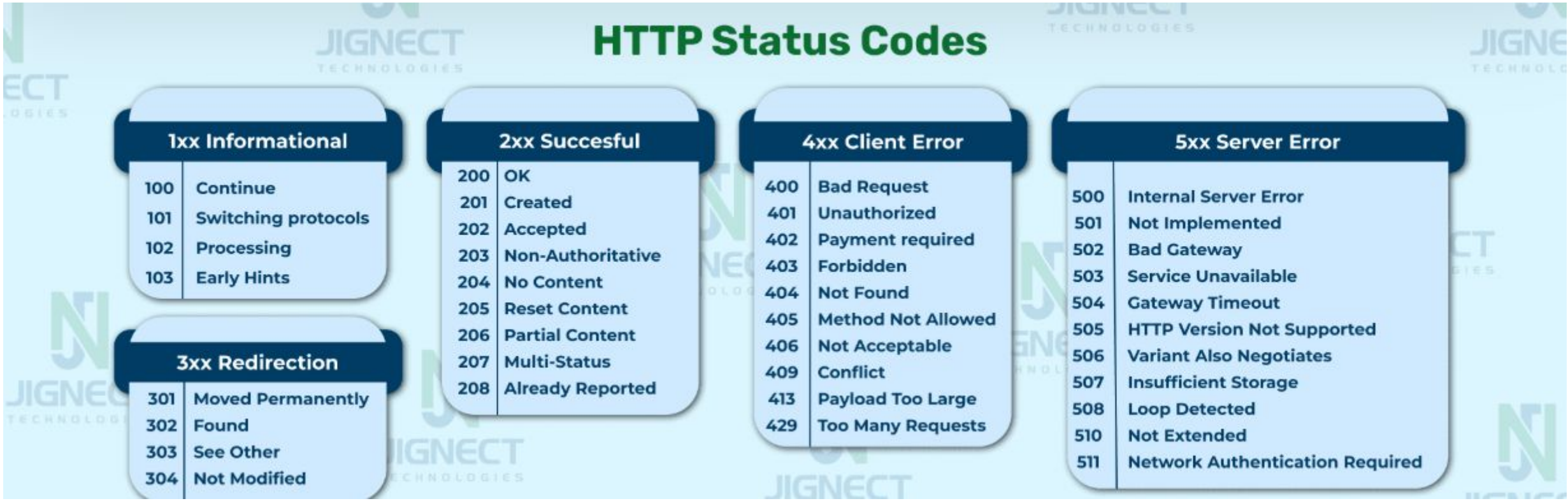





Login
Username
Password
Login
No Account ? Register Here


Register
Username
Email
Password
Confirm Password
Sign Up
Have an Account ? Login Here


Create form
handleChange
handleSubmit



https://jignect.tech/http-status-codes-understanding-their-significance-and-importance/

200 Ok (update/ delete success)
201 create success

/routes/authen.j

```
const express = require('express');
const router = express.Router();
const bcrypt = require('bcrypt');
const prisma = require('../prisma/prisma')

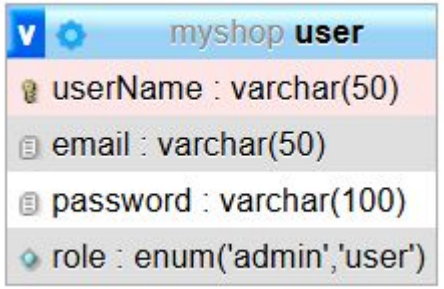
router.post('/register', async (req, res) => {

});

router.post('/login', async (req, res) => {

});

module.exports = router;
```



register

```
router.post('/register', async (req, res) => {
    const { userName, email, password } = req.body;
    const getUser =

    //get duplicate username or password user



    if (getUser.length > 0) {
        res.status(409).json({ message: "Duplicate Username or Email" });
    } else {
        const hashPassword = await bcrypt.hash(password,10)
        const newUser = await prisma.user.create({

        //create new user


        });
        res.status(201).json({  message: 'User created successfully'});
    }
});
```

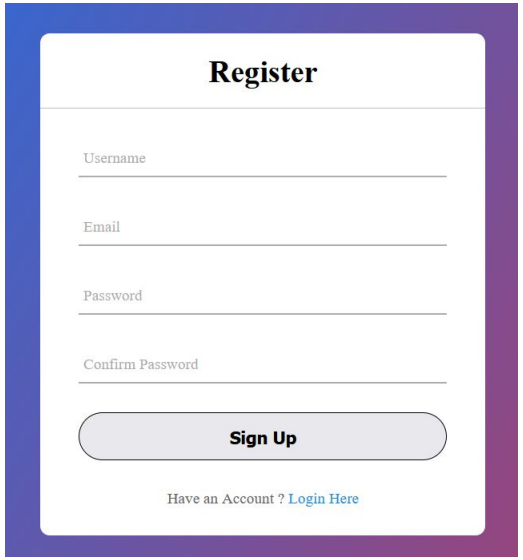
```jsx
import { useState } from 'react'
import Axios from 'axios'
import { useNavigate } from 'react-router-dom'

import './register.css'

function Register() {
    const [inputs, setInputs] = useState(_____________________________)
    const navigate = useNavigate();
    function handleChange(e) {                    }
    const handleSubmit = async (e) => {                  }

    return (
        <form>

            <div>
                Have an Account ? <a href="/login">Login Here</a>
            </div>
        </form>
    );
}
export default Register
```



```
const handleSubmit = async (e) => {
    e.preventDefault();

    if (inputs.password != inputs.cpassword) {
        alert("Password mismatch")
    }
    else {
        try {
            const response = await Axios.post(_____________________________, inputs)
            if (response.status === 201) {
                alert(response.data.message)
                navigate('/login')
            }
        } catch (err) {
            if (err.response.status === 409) {
                alert(err.response.data.message)
            }
        }
        setInputs(            )
    }
}
```

```
router.post('/login', async (req, res) => {
    const { userName, password } = req.body;

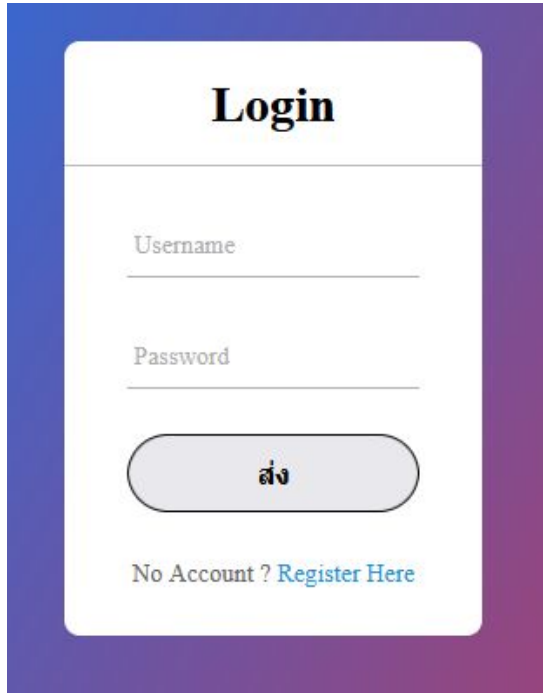
    const getUser = await prisma.user.findUnique( //for unique field


    );
    if (!getUser) {
        res.status(404).json({ message: "Unknown Username" });
    } else {
        const isMatch = await bcrypt.compare(password, getUser.password)
        if (!isMatch) {
            res.status(404).json({ message: "Password Incorrect" });
        }
        else {
            res.status(200).json({ message: "Login Success" });
        }
    }
});
```

```
findMany  => getUser.length==0
          => getUser[0]
```

```
const handleSubmit = async (e) => {
        e.preventDefault();

        try {
            const response = await Axios.post(_____________________________, inputs)
            if (response.status === 200) {
                alert(response.data.message)
                navigate('/home')
            }
        } catch (err) {
            if (err.response.status === 404) {
                alert(err.response.data.message)
            }
        }
        setInputs({ userName: "", password: "" })
    }
```

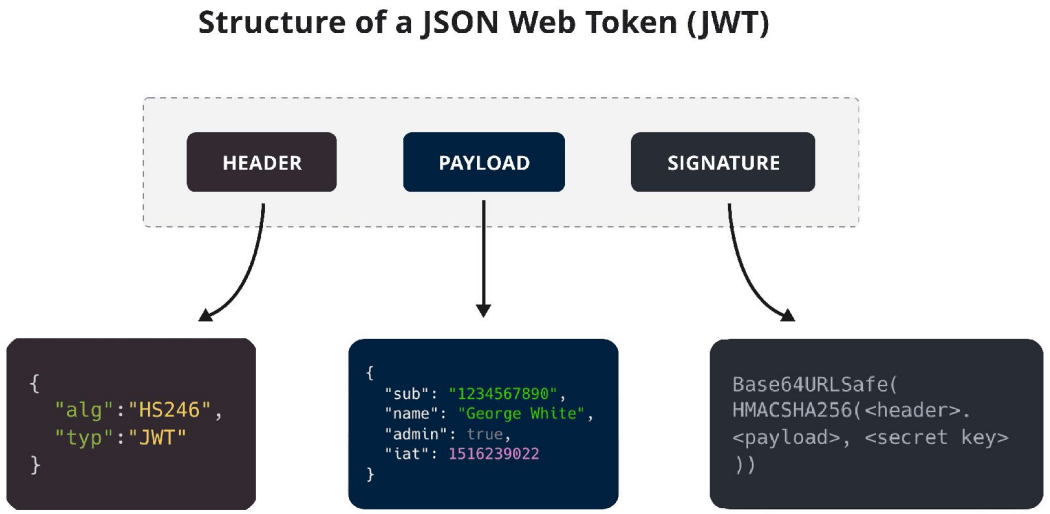


```
npm install jsonwebtoken
```

JSON Web Token (JWT)

JWT authentication and session authentication are ways to authenticate users of your web app

JWT is a commonly used stateless user authentication standard used to securely transmit information between client and server in a JSON format.



Header

consists of token type, which is JWT, and the signing algorithm used

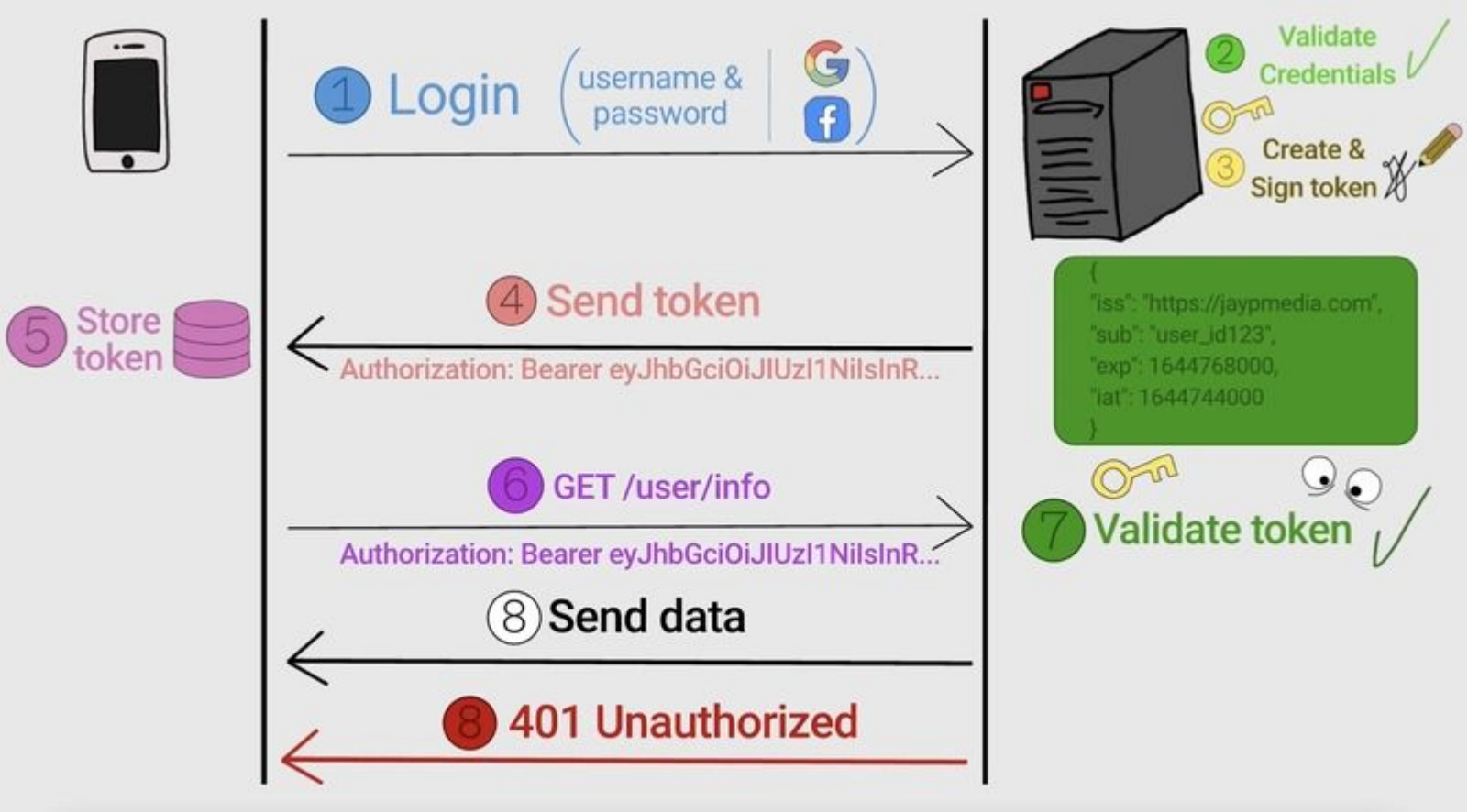
Payload

consists of the claims - statements about the user, and additional data.

Signature

It is calculated using the header, the payload, and the secret, which are fed to the signing algorithm to use.

JWT Steps



When the client wants to access a protected resource on the server, it sends the JWT in the Authorization header of the HTTP request.

```
axios.get(URL, {
   headers: {
      'Authorization': 'Bearer ' + token,
   },
})
```

https://dev.to/jaypmedia/jwt-explained-in-4-minutes-with-visuals-g3n

JWT example

```
const jwt = require('jsonwebtoken');

const payload = {
    userId: 'user123',
    username: 'john.doe',
    role: 'admin'
};

const secretKey = 'your_super_secret_key’;
// In production, store this securely (e.g., environment variable)
```

```
const token = jwt.sign(payload, secretKey, { expiresIn: '1h' });
console.log(token);
```

```
{ expiresIn: '1m' }
```

.env

```
JWT_KEY="jwt-secret-key"
```

/authen/login

```
        if (!isMatch) {    }

        else { //login success

            const token = jwt.sign(
               { userId: getUser[0].userId },
               process.env.JWT_KEY,
               { expiresIn: '1m’ }
            )
            return res.status(200).json({ message: "Login Success", token: token })
```

3

4

Login.jsx

```
      if (response.status === 200) {
        alert(response.data.message)
        localStorage.setItem('token',response.data.token)
        navigate('/')
      }
```

5

routes/authen

```
const verifyToken = (req, res, next) => {
    try {
        const token = req.headers['authorization'].split(' ')[1];
        if (token == "null") {
            return res.status(403).json({ message: "no token provide" })
        }
        else {
            const decode = jwt.verify(token, process.env.JWT_KEY)
            req.userId = decode.userId;
            next()
        }
    } catch (err) {
        return res.status(401).json({ message: "Invalid token" })
    }
}
```

routes/authe

```
router.get('/home', verifyToken, async (req, res) => {
    try {
        const userId = req.userId

        const getUser = await prisma.user.findMany({
            where: {
                userId: userId,
            },
        });

        if (getUser.length == 0) {
            return res.status(404).json({ message: "User doesn't exist" });
        }
        return res.status(200).json({ user: getUser })
    } catch {
        return res.status(500).json({ message: "server error" })
    }
})
```

Home.jsx

```
import ...
function Home() {
  const [userInfo, setUserInfo] = useState({});
  const navigate = useNavigate()
  useEffect(() => {
    const fetchData = async () => {
      const token = localStorage.getItem('token')
      try {
        const response = await Axios.get('http://localhost:3000/authen/home', {
          headers: {
            "authorization": `Bearer ${token}`
          }
        })
        if (response.status != 200 && response.status != 201) { navigate('/login')
        } else { setLoginUser(response.data.user[0]);
        }
      } catch (err) {
        navigate('/login')
      }
    };
    fetchData(); // Call the async function
```

6

8

Real-time data updates

```
useEffect(() => {
  const fetchData = async () => {



  };
  fetchData(); // Call the async function



  // Set up polling interval (e.g., every 5 seconds)
  const intervalId = setInterval(fetchData, 5000);



  // Clean up the interval when the component unmounts
  return () => clearInterval(intervalId);
```

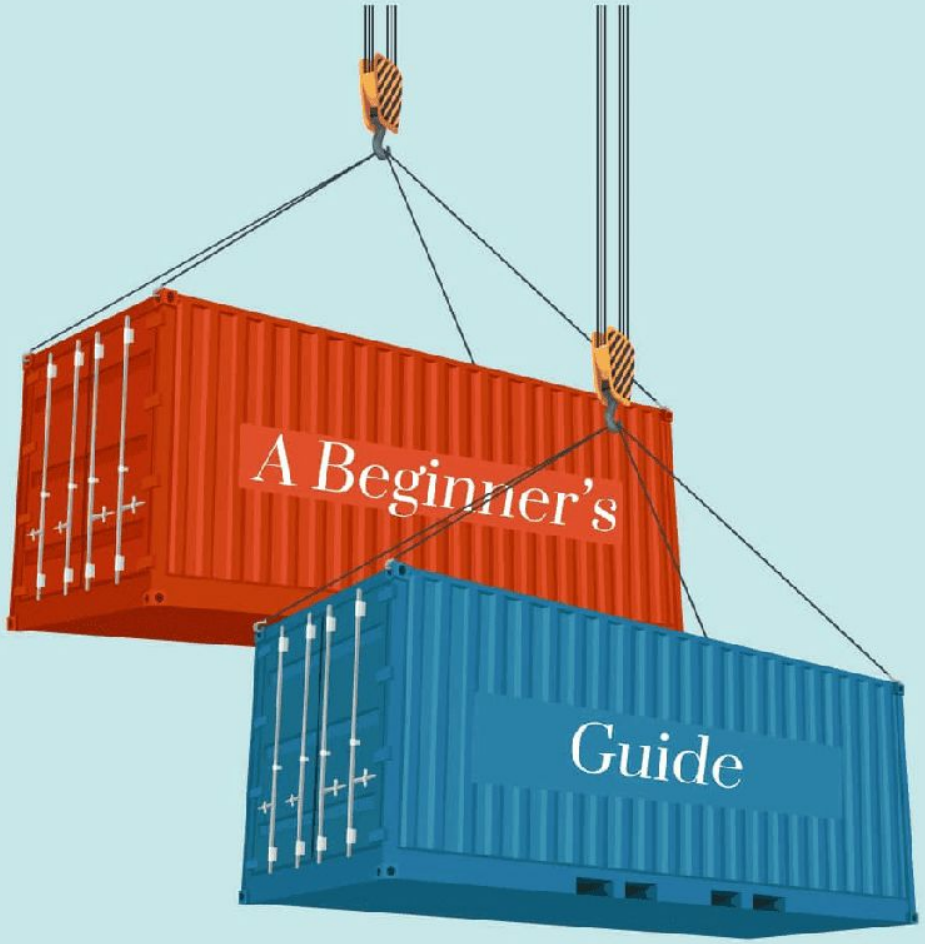
Logout

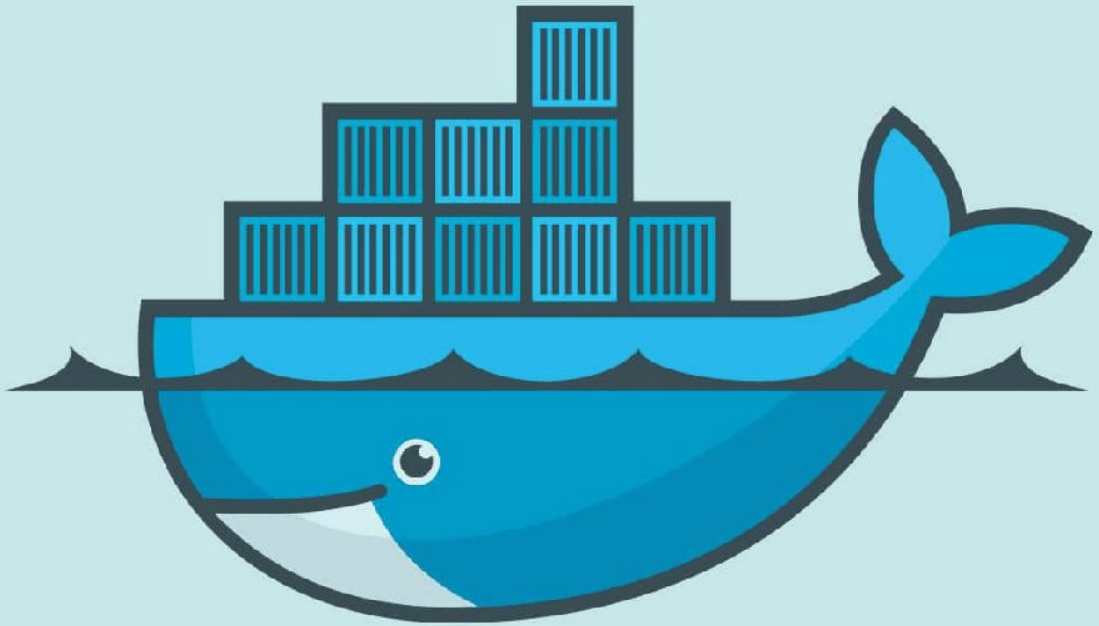
```
const handleLogout = () => {
    // Clear authentication tokens
    localStorage.removeItem('token');


    // also clear other user-specific data if stored locally
    // localStorage.removeItem('userProfile');


    // Redirect to the login page or home page
    navigate('/login');
  };
```

```
<button onClick={handleLogout}>Logout</button>
```





docker

Open Replay

Docker installation

Install -> Docker Desktop



Check version -> docker --version

```
C:\Users\user>docker --version
Docker version 29.8.0, build 88096ef
```



prepare project



Dockerfile

```
# Base Image
FROM node:18-alpine
# Work directory
WORKDIR /app
# copy file to Container
COPY . .
# expose port 3000
EXPOSE 3000
# run application
CMD ["node", "app.js"]
```

package.json

```
{
  "name": "docker-demo",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {}
}
```

app.js

```
const http = require('http');
const hostname = '0.0.0.0';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello from Docker!\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Create Docker Image

Open docker desktop

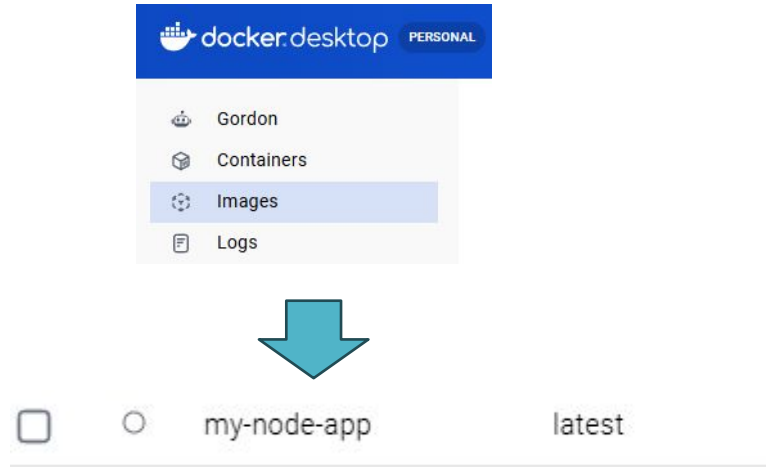
Build image

```
>docker build -t my-node-app .
```

Check image

```
>docker images
```

```
D:\workspace\vscode\dockerTest>docker images
IMAGE                  ID            DISK USAGE   CONTENT SIZE   EXTRA
my-node-app:latest     f4f9bc7847e3       181MB         44.9MB
```



Delete image

```
>docker rmi my-node-app
```

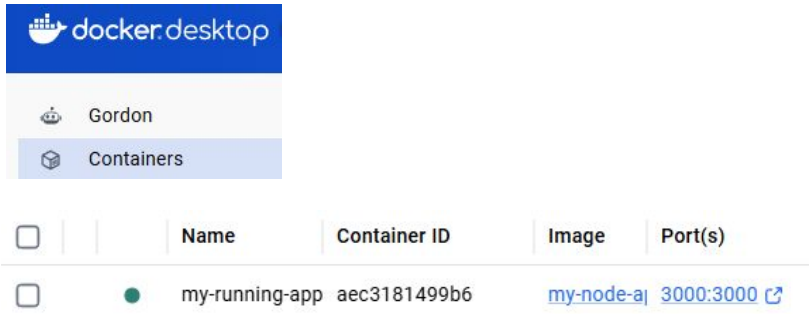
Start container

```
>docker run -d -p 3000:3000 --name my-running-app my-node-app
```

-d: run in Background
-p 3000:3000: map port 3000 in server with 300 in container
--name: set Container name to my-running-app

http://localhost:300

Hello from Docker!



Stop container

```
>docker stop my-running-app
```

Delete container

```
>docker rm my-running-app
```

Stop+Delete

```
docker rm -f my-running-app
```

Export docker

1. Export docker to .tar

```
docker save -o my-node-app.tar my-node-app
```

2. Load docker

```
docker load -i my-node-app.tar
```

Multi-container



Add Dockerfile in frontend and backend
Add docker-compose.yml to project Folder

/frontend/Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . ./
RUN npm run build


FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

/backend/Dockerfile

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . ./
EXPOSE 5000
CMD ["node", "src/server.js"]
```

docker-compose.yml

```yaml
version: '3.8'
services:
  backend:
    build: ./backend
    container_name: backend_service
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      -
DATABASE_URL=mysql://root:password@host.docker.internal:3306/myshop

  frontend:
    build: ./frontend
    container_name: frontend_service
    ports:
      - "81:80"
    depends_on:
      - backend
```

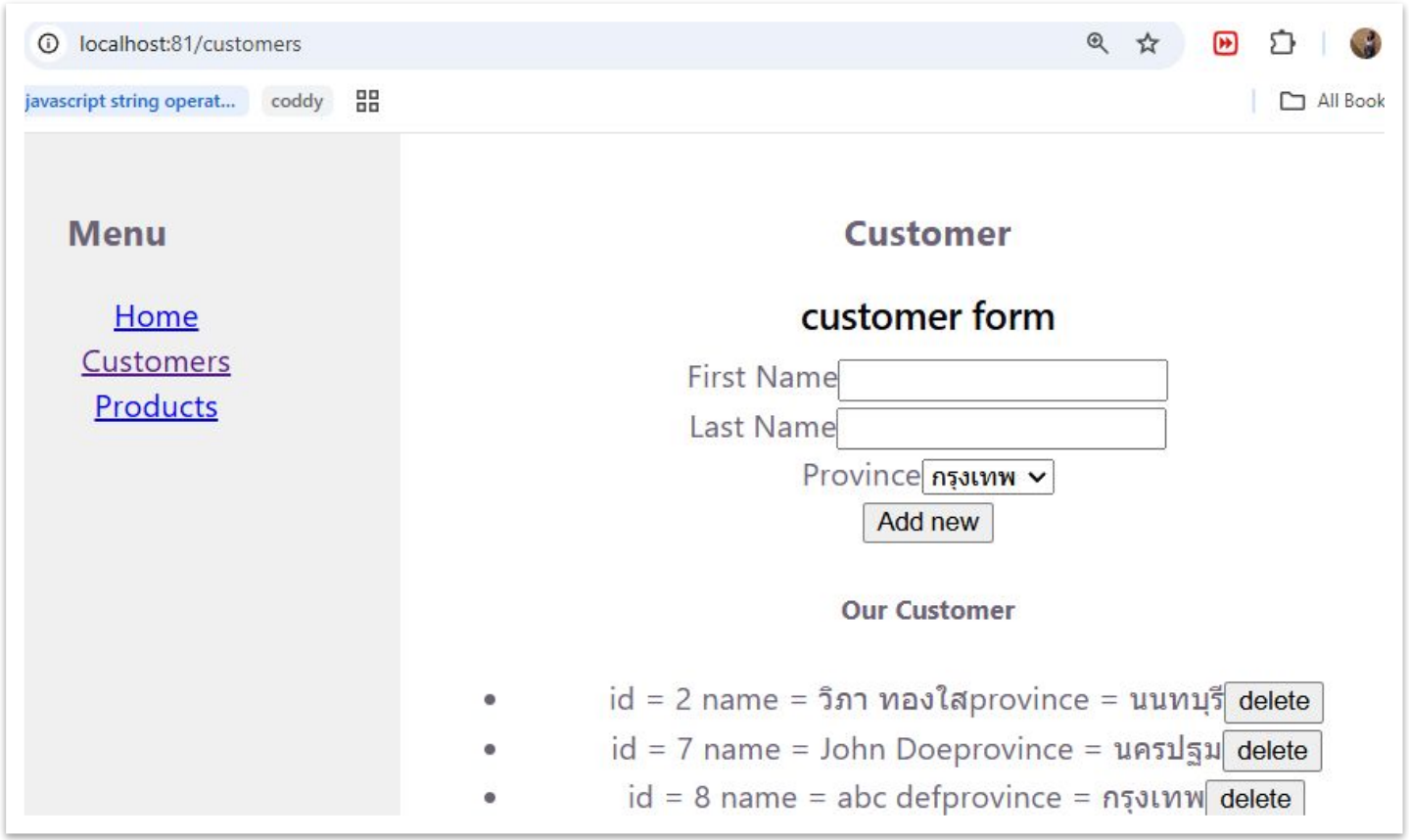
Backend/.env

```
DB_HOST=host.docker.internal
DB_PORT=3306
DB_USER=root
DB_PASSWORD=password
DB_NAME=myshop

PORT=3000

DATABASE_URL="mysql://root:password@host.docker.internal:3306/myshop
"
```

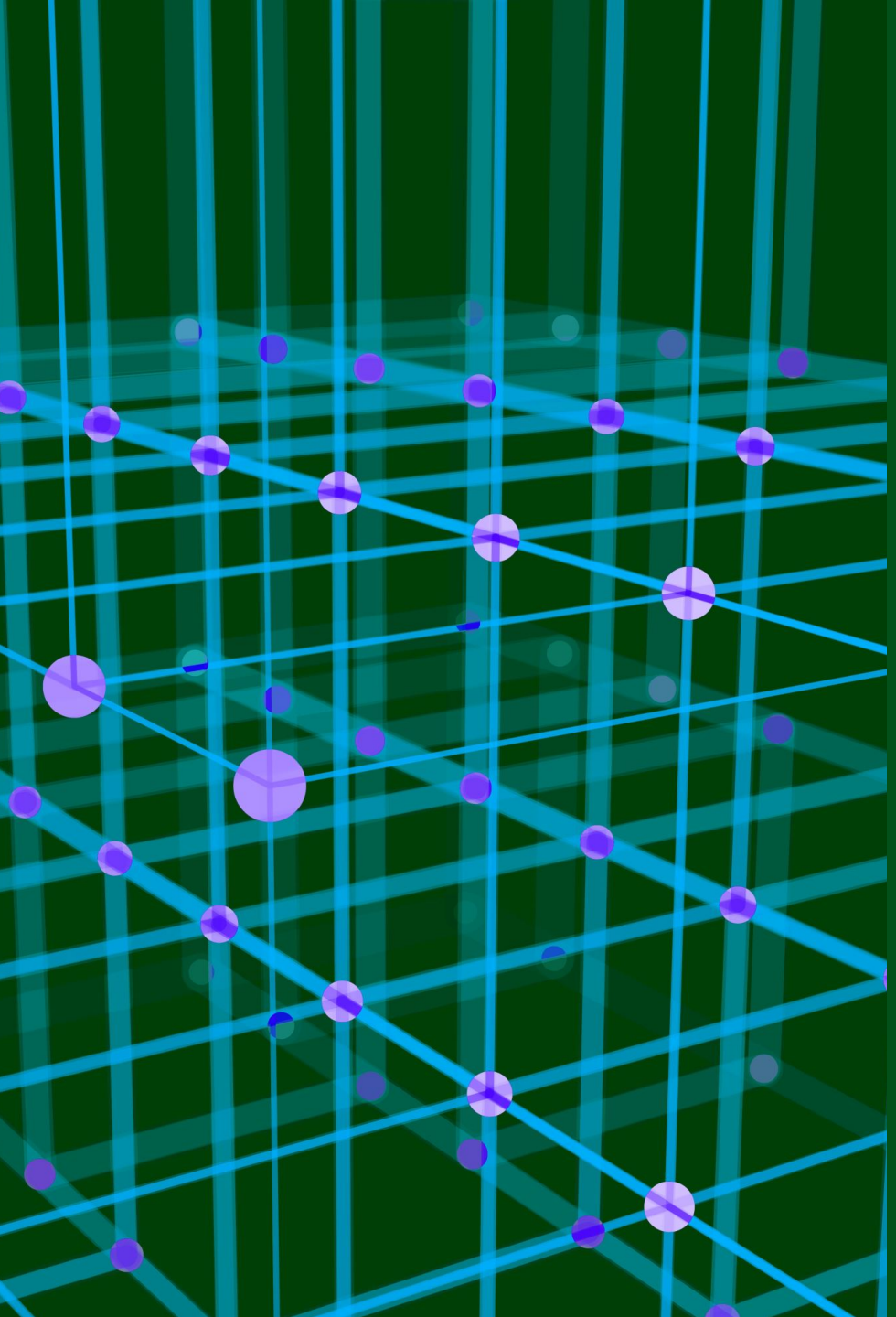
Start service `>docker compose up -d --build`





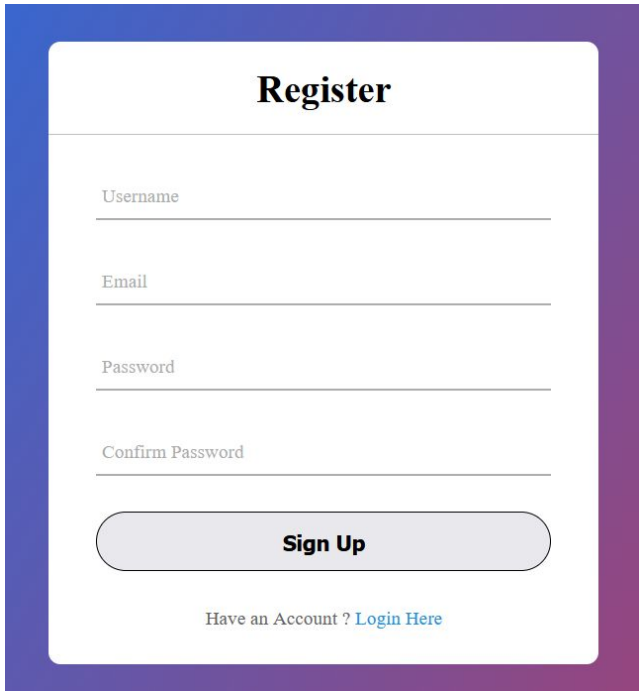
Stop service `docker compose down`

Stop and Remove volume `docker compose down -v`

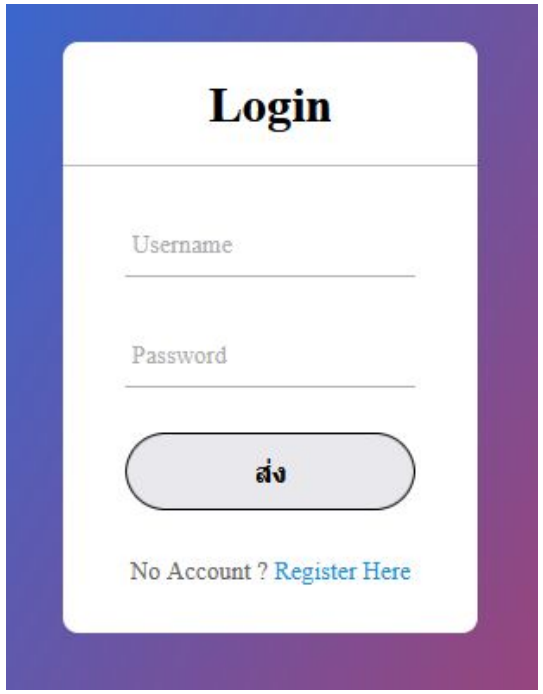


Todo

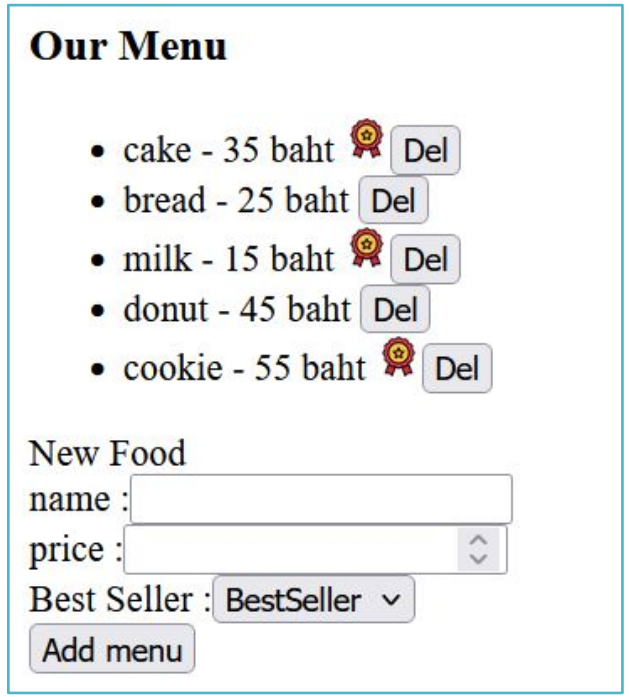
Register page



Login page



Home



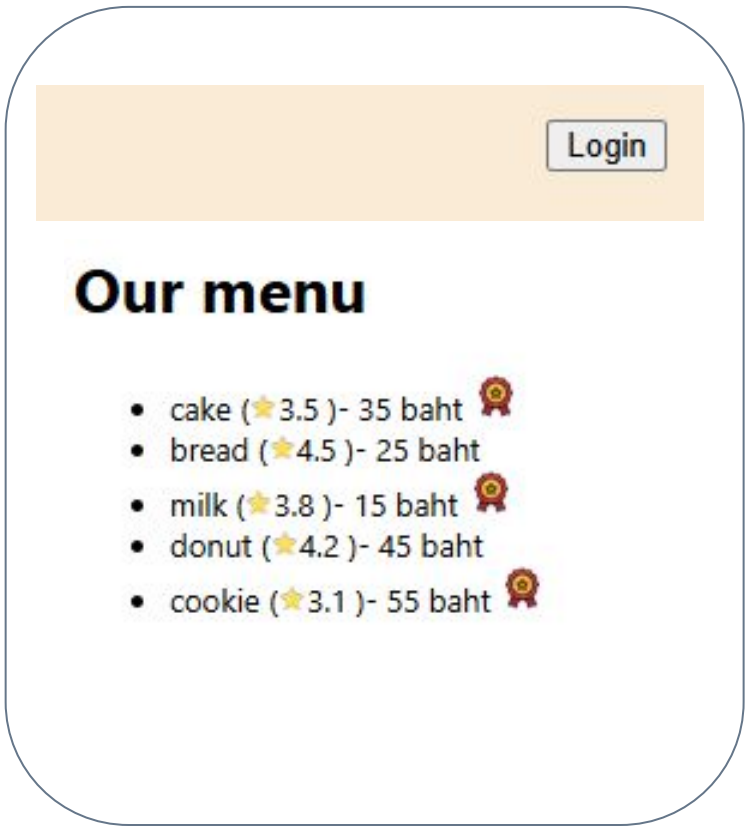
Home

Guest mode
- login

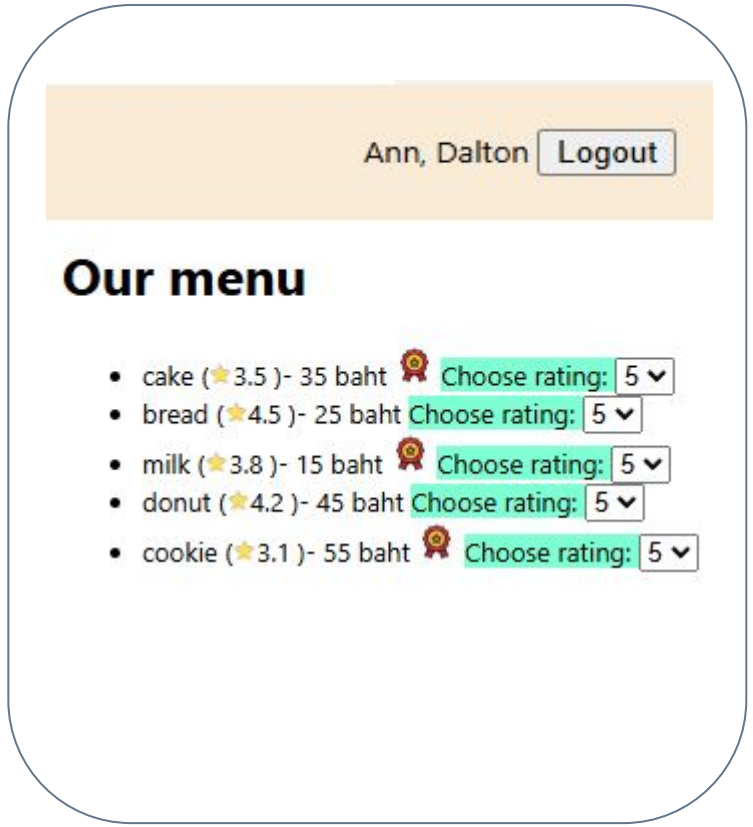
User mode
- rating
- logout

Admin mode
- Addproduct
- Change user role
- logout

Guest mode

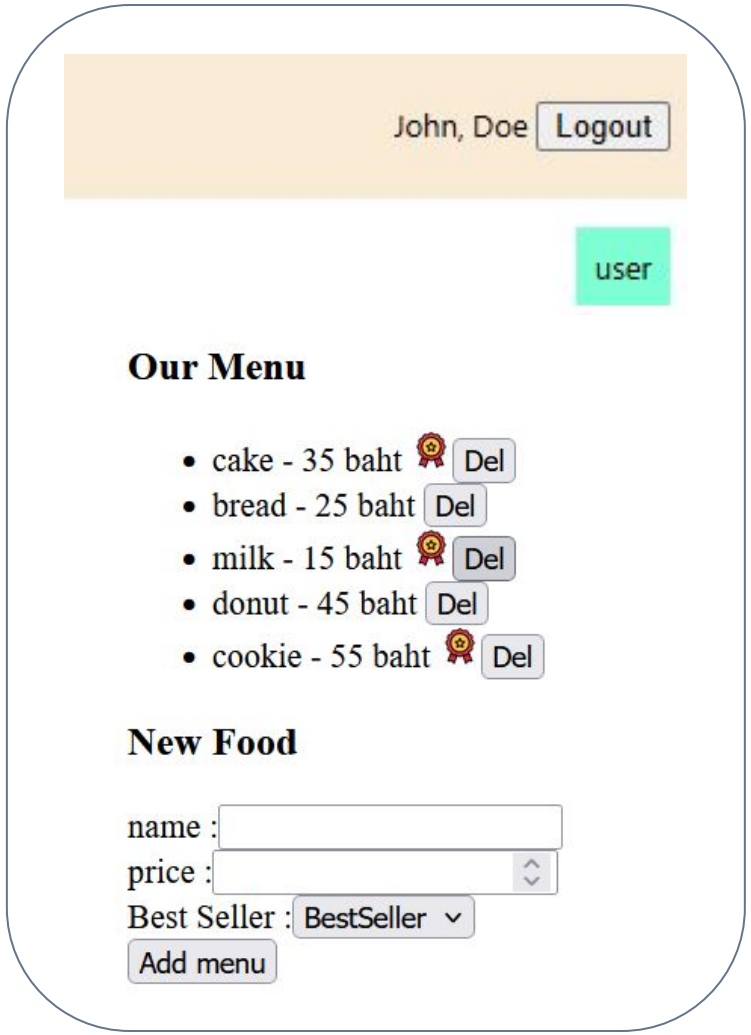


user mode

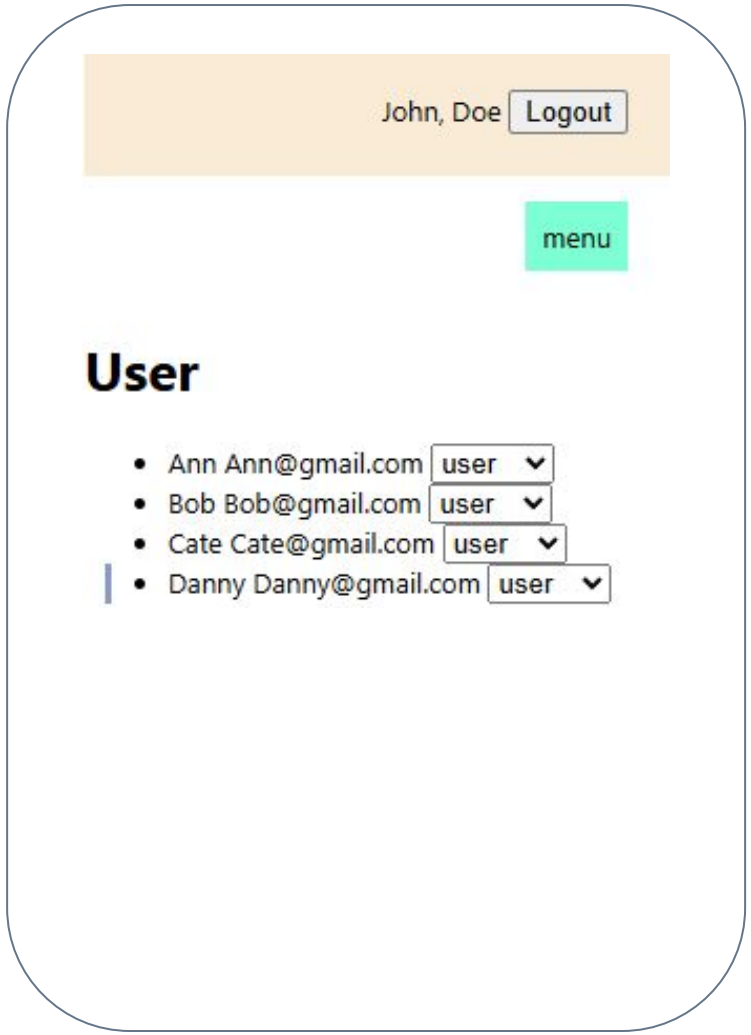


* Menu show with real-time data update

Admin mode


John, Doe Logout
user
Our Menu
cake - 35 baht Del
bread - 25 baht Del
milk - 15 baht Del
donut - 45 baht Del
cookie - 55 baht Del
New Food
name :
price :
Best Seller : BestSeller
Add menu




John, Doe Logout
menu
User
Ann Ann@gmail.com user
Bob Bob@gmail.com user
Cate Cate@gmail.com user
Danny Danny@gmail.com user